Mamba for Side-Channel Attacks: Bidirectional State-Space Encoders Against Masked AES

Beatrise Bertule
beatrise.bertule@ru.nl
Radboud University Nijmegen
Nijmegen, The Netherlands

Lizzy Grootjen
lizzy.grootjen@ru.nl
Radboud University Nijmegen
Nijmegen, The Netherlands

Abstract

Profiled side-channel attacks (SCAs) recover secret cryptographic keys by first characterizing the leakage of a device under the attacker's control and then using that model to attack a similar target. Classical approaches such as template attacks (TAs) build explicit statistical models of the leakage, while deep-learning methods instead learn this mapping directly from raw power traces. Because informative leakage points are spread across a wide window along the trace, effective models have to capture long-range context. To model this temporal dependency, we investigate the Mamba block, a selective state-space model that processes sequences in linear time while propagating information selectively across long ranges. Our model stacks Mamba blocks into bidirectional encoders with residual connections, enabling it to capture temporal context in both forward and backward directions along a power consumption trace. Experimental results on protected, masked ASCADv1 dataset show that the Mamba-based model successfully recovers the secret key with fast guessing-entropy convergence. While the proposed model is competitive with a parameter-matched MLP baseline, it does not outperform it, suggesting that on these relatively simple, fixed-key datasets the additional architectural complexity of Mamba offers no measurable advantage, while its state-space formulation remains a promising direction for longer, noisier, and more strongly protected traces.

CCS Concepts

• **Security and privacy → Side-channel attacks and countermeasures**.

Keywords

Side-Channel Attacks, Deep Learning, Mamba, State-Space Models, AES, Profiled Attacks, Neural Networks, Bidirectional Encoders, ASCAD, Hardware Security

ACM Reference Format:
Beatrise Bertule and Lizzy Grootjen. 2026. Mamba for Side-Channel Attacks: Bidirectional State-Space Encoders Against Masked AES. In . ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Cryptographic algorithms such as the Advanced Encryption Standard (AES) [15] are designed to guarantee confidentiality. However, their physical implementations can leak information through side channels which are measurable physical phenomena such as power consumption [11], electromagnetic emissions [21], or timing [10]. While cryptanalysis targets theoretical weaknesses of an algorithm, side-channel attacks (SCAs) exploit physical leakage to recover secret information even when the algorithm is mathematically secure [22].

Among SCAs, profiled attacks are considered the strongest, because they assume an adversary who first characterizes the leakage of a device under full control and then applies that characterization to a target device [4]. Template attacks (TAs), which model the leakage of each class with a mean vector and a covariance matrix, were the first attacks of this type [4]. Since profiling is a supervised classification problem, deep learning (DL) replaced templates as the classification step and now defines the state-of-the-art [20]. In deep learning-based SCA (DL-SCA), a neural network is trained on traces from the profiling device to predict an intermediate value of the cipher, typically the output of the S-box, and the resulting class probabilities are accumulated over attack traces to rank key candidates. DL-SCA learn discriminative features directly from raw traces, which removes the need for manual point-of-interest selection [13, 16], and they tolerate trace misalignment and countermeasures that defeat classical templates [9, 14]. Multilayer perceptrons (MLPs) and convolutional neural networks (CNNs) are the two most commonly applied architectures [9, 14, 16, 25], more recently combined with attention mechanisms [6, 8, 13].

Despite extensive exploration of conventional architectures, sequence oriented models, such as Mamba, explicitly designed to capture long range dependencies remain largely unexplored in the context of SCA. Power traces are inherently sequential, and leakage caused by a sensitive computation can be temporally spread across many samples [14]. Architectures that explicitly model temporal context may therefore better capture distributed leakage patterns. This work explores whether Mamba [7], a selective state-space model, can be effectively adapted for profiled SCA. Mamba extends classical state-space models with a selectivity mechanism that lets the model retain informative components of a sequence while suppressing noise.

In this paper, we adapt Mamba to profiled SCA on a protected implementation of the AES-128 algorithm. We design a Mamba-based network with bidirectional encoders: a forward encoder models dependencies in the original temporal direction, while a backward encoder captures dependencies in the reverse direction. The motivation is that a power trace is not a causal stream like a sentence:

the full measurement is available offline, so there is no reason to restrict the model to a left-to-right view. A unidirectional encoder compresses the sequence in one direction only, so leakage appearing late in the trace cannot inform earlier positions. Reading the trace from both ends removes this restriction and lets the model relate leakage points regardless of the order in which they occur, which is the situation created by masking. The model is trained to map power traces to the AES S-box output, which is then used to recover the secret key byte. Its effectiveness is evaluated with key rank and guessing entropy [20] on a protected dataset, with an MLP from prior work as the baseline.

The main contributions of this work to DL-SCA are as follows:

(1) We propose a bidirectional Mamba architecture for profiled SCA, built from stacked Mamba blocks with residual connections.
(2) We explore the application of the Mamba architecture, specifically the selective state-space mechanism, to explicitly model temporal context within power traces.
(3) We assess the proposed Mamba-based model on a protected implementation of AES, side-channel leakage datasets, and benchmark its performance against an MLP model, a successful model proposed by prior research.

2 Background

In this section we introduce the leakage model, target intermediate value, attack types (non-profiled vs. profiled), and evaluation metrics used in this work. We then describe the Mamba architecture and its selective state-space mechanism.

2.1 Side-Channel Leakage and Leakage Models

Physical leakage L is commonly modeled as a deterministic component plus noise : $L = \delta(V_k) + B$, where δ denotes the unknown deterministic leakage function, V_k corresponds to a sensitive intermediate value that depends on the secret key k, and B represents additive noise [5]. Since δ is unknown, an attacker approximates it with a leakage model m. Two common leakage models are the Hamming Weight (HW) model, $m_{HW}(v) = \sum_i b_i$, and the Identity (ID) model, $m_{ID}(v) = v$, which treats every value as a distinct class. In this work, we apply the ID leakage model.

AES processes data byte-wise, so attacks follow a divide-and-conquer strategy and recover one key byte at a time; for research purposes, recovering a single key byte is sufficient to demonstrate attack effectiveness [20]. We target the output of the first-round S-box for a key byte b:

$$y = S\text{-box}\big[p[b] \oplus k[b]\big]. \tag{1}$$

2.2 Profiled and Non-Profiled Attacks

Side-channel attacks fall into two families that differ in what the adversary is assumed to have access to before the attack [19].

A *non-profiled* attack operates directly on the target device without a preparation phase: for each key candidate it maps the hypothetical intermediates $v^{k_i} = f(p, k_i)$ to hypothetical leakage through a leakage model m and compares these against the measured traces with a statistical distinguisher, as in differential power analysis (DPA) [11]. Its success depends on how well m matches the device and usually requires many traces measured on the target itself.

A *profiled* attack splits the effort into two phases and is considered the strongest type of side-channel attack [4]. In the profiling phase, the adversary uses a clone device, over which they have full control, to characterize the leakage associated with each possible value of the targeted intermediate. In the attack phase, this characterization is applied to traces measured on the target device, whose key is unknown. The characterization step absorbs most of the difficulty, so few traces from the target are needed afterwards — a single trace would suffice if the characterization were perfect. While this is not always realistic in practice, it is precisely what makes the profiled setting the appropriate one for security evaluation, as it bounds the worst-case attack scenario.

The template attack was the first profiled attack [4]. It describes the leakage of each class with a mean vector and a covariance matrix and classifies an attack trace by maximum likelihood. Because profiling is a supervised classification problem, machine l2earning is a natural replacement for templates: rather than assuming a parametric leakage distribution, the classifier learns the mapping from a trace to the intermediate value from labeled examples [20]. The leakage model then enters only through the choice of labels.

For a deep-learning profiled attacker on AES or similar block ciphers, the trained model outputs a probability distribution P_j over S-box outputs for each attack trace j. Given the known plaintext byte p_j, the hypothetical label under key candidate k_i is $h(k_i) = S\text{-box}[p_j \oplus k_i]$. Candidate scores are accumulated in log-space over N attack traces,

$$s(k_i) = \sum_{j=0}^{N} \log\big(P_j(h(k_i))\big), \tag{2}$$

and sorting these scores in descending order yields a key-guessing vector g.

2.3 Evaluation Metrics

The *key rank* is the position of the correct key byte in the key-guessing vector g. *Guessing entropy* (GE) is the average key rank over M repeated attacks on random trace subsamples,

$$GE = \frac{1}{M} \sum_{j=0}^{M} \text{rank}_j(g). \tag{3}$$

An attack succeeds when GE reaches zero, and it is more efficient when it reaches zero with fewer attack traces. GE convergence — how GE decreases as more attack traces are used — therefore indicates both how quickly an attack becomes effective and the number of traces required in practice. We favor GE and key rank over classification accuracy as an evaluation metric as plain classification accuracy correlates poorly with key-recovery success [18].

2.4 Mamba

Mamba [7] is built on selective State-Space Models (SSMs). A continuous SSM maps a 1-D input $x(t)$ to output $y(t)$ through a latent state $h(t) \in \mathbb{R}^N$ via

$$h'(t) = Ah(t) + Bx(t), \qquad y(t) = Ch(t), \tag{4}$$

with $A \in \mathbb{R}^{N\times N}$, $B \in \mathbb{R}^{N\times 1}$, $C \in \mathbb{R}^{1\times N}$, where the state equation $h'(t)$ describes how the latent state evolves over time. To operate on discrete sequences, A and B are discretized with step size Δ, $\bar{A} = \exp(\Delta A)$, $\bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I) \cdot \Delta B$, giving the discrete recurrence

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \qquad y_t = Ch_t. \tag{5}$$

Traditional SSMs are time-invariant: A, B, C are fixed for every token, which limits content-awareness and forces the model to compress the entire sequence history with uniform weights. Selective SSMs instead make B, C, and the discretization step Δ input-dependent functions of the current token,

$$B_t = s_B(x_t), \quad C_t = s_C(x_t), \quad \Delta_t = \tau_\Delta(s_\Delta(x_t)), \tag{6}$$

where s_B, s_C are linear projections, s_Δ is a linear projection broadcast over the feature dimension, and τ_Δ is the softplus function. This enables content-aware, selective compression of sequence history: a large Δ_t lets the model focus on the current input and effectively reset the state, while a small Δ_t preserves prior state and attenuates the current input. Because this breaks the convolutional formulation of classical SSMs, Mamba relies on a hardware-aware parallel scan (kernel fusion, selective scan, recomputation) to remain efficient.

A Mamba block expands an embedded sequence of dimension D to a higher-dimensional space via a linear projection, applies a 1-D convolution to capture local dependencies, and — after a SiLU/Swish activation — passes the result through the selective SSM (S6) layer, which achieves context-awareness via the dynamically generated (Δ, B, C). In a parallel branch, the expanded sequence passes through a separate activation acting as a multiplicative gate that modulates the SSM output. The gated result is projected back to dimension D. Multiple blocks can be stacked with residual connections, and the design is intended as a linear-time replacement for the attention + MLP stack found in transformer models.

3 Related Work

Since the first systematic study of DL for SCA in 2016 [14], the field has grown to cover architecture choice [1, 9], dataset design [3], feature engineering [16, 17], hyperparameter optimization [2, 24], and evaluation metrics [18]. CNNs and MLPs remain the most widely adopted architectures and are frequently used as simple baselines to isolate the effect of optimization choices [1, 23].

More recently, hybrid architectures combining CNNs with attention mechanisms have been proposed to jointly capture local patterns and long-range dependencies. Lu et al. [13] combine lightweight CNN/locally-connected layers with bidirectional LSTMs and attention to focus on informative time steps. Feng et al. [6] propose HACNN-SCA, combining convolution with a hybrid channel-and-spatial attention module. He et al. [8] propose AMCNNet, using multi-scale convolution together with self-attention over enhanced channels. All three emphasize relevant leakage regions and suppress noise, improving over plain CNN/MLP baselines.

Attention, however, becomes computationally expensive for long traces. Mamba [7] offers an alternative with linear-time complexity. To date, only one prior work has applied Mamba to SCA: Li et al. [12] propose a CNN-Residual-Mamba hybrid with a unidirectional stack of three residual Mamba blocks. In contrast, our model explicitly captures *bidirectional* long-range dependencies via parallel forward and backward Mamba encoders, which is a configuration not explored in prior work.

4 Experimental Setup

In this section, we first describe our assumed attacker model. Then we describe the dataset, and then we present the two architectures we compare and the hyperparameter search that produced their configurations. Finally, we give the training protocol and the procedure used to evaluate the resulting attacks.

4.1 Attacker Model

The attacker's goal is to recover the secret key k^* used by an AES-128 implementation running on a target device, using only the power consumption measured while the device encrypts. We assume the standard profiled setting described in Section 2.2. This section states the assumptions behind that setting and their motivation. Our attacker model makes the following assumptions:

(1) The adversary measures the power consumption of the device during encryption. The adversary does not inject faults, does not modify the device, and does not read its internal memory.
(2) The adversary controls a device of the same type as the target, chooses both the plaintext and the key on that device, and runs the implementation as often as required. The number of profiling traces is large but finite.
(3) The adversary knows which algorithm the device runs and which intermediate value to target, but not the secret key of the target device, and the plaintext input associated with each attack trace is observable.
(4) The recorded traces cover the targeted operation. The dataset used in this work provides preselected windows around the first encryption round, and no further realignment is applied.

4.2 Dataset

We evaluate on the fixed-key variant of ASCADv1 [3]. The dataset contains power traces of a first-order Boolean-masked AES-128 implementation running on an 8-bit ATMega8515 microcontroller. Each trace is pre-aligned and cropped to a window of 700 samples around the first-round S-box computation, so neither realignment nor feature selection is applied on our side; this matches the trace-alignment assumption of our attacker model (Section 4.1). We attack byte 2, the first masked byte of the implementation, since bytes 0 and 1 are left unprotected. Labels follow the ID leakage model,

$$y_j = S\text{-box}\left[p_j \oplus k_j^*\right], \tag{7}$$

which gives $K = 256$ classes, encoded as one-hot vectors. We use 50 000 profiling traces and 10 000 attack traces. Traces are standardized feature-wise, with the mean and the standard deviation computed on the profiling set only, so that no statistic derived from the attack set enters training.

4.3 Model Architectures

Both models take a trace of $T = 700$ samples as input and produce logits over the $K = 256$ classes. They are matched in learnable-parameter count, so that a difference in attack performance can be attributed to the architecture rather than to model capacity.

Algorithm 1 Bidirectional Mamba Forward Pass

Require: Raw signal $x \in \mathbb{R}^{B\times T}$
Ensure: Logits $y \in \mathbb{R}^{B\times K}$
$x \leftarrow \text{Reshape}(x, [B, 1, T])$
$z \leftarrow \text{Conv1d}(x; D_{model}; \theta_{conv})$
$z \leftarrow \text{Reshape}(z, [B, L, D_{model}])$
$z_{fwd} \leftarrow z; \quad z_{bwd} \leftarrow \text{Flip}(z, \text{dim} = 1)$
for $i \leftarrow 1$ **to** N_{blocks} **do**
  $z_{fwd} \leftarrow \text{Block}^{(i)}_{fwd}(z_{fwd})$ ▷ forward Mamba blocks
  $z_{bwd} \leftarrow \text{Block}^{(i)}_{bwd}(z_{bwd})$ ▷ backward Mamba blocks
end for
$h_{fwd} \leftarrow z_{fwd}[:, L, :]; \quad h_{bwd} \leftarrow z_{bwd}[:, L, :]$
$h \leftarrow \text{Concat}([h_{fwd}, h_{bwd}], \text{dim} = -1)$
$y \leftarrow \text{Linear}(h; \theta_{net})$
return y

MLP baseline. We use a multilayer perceptron as the baseline for two reasons. It is the architecture that Perin et al. [16] report as the strongest on this dataset, and a fully connected layer combines all samples of a trace at once, which reproduces the effect of a higher-order attack and makes the MLP a natural fit for masked implementations [9]. We follow their design: an input layer mapping T features to $N_{neurons}$ units, N_{hidden} hidden layers of $N_{neurons}$ units with SELU activations, and a linear output layer that produces logits over the 256 classes, trained with cross-entropy loss.

Mamba model. Our proposed architecture (Figure 1, Algorithm 1) consists of three modules:

Convolution module. A 1-D convolution with kernel size k and stride s embeds the raw trace into a sequence of $L = \lfloor (T - k)/s \rfloor + 1$ feature vectors of dimension D_{model}.

Bidirectional Mamba module. Two parallel encoders, each a stack of N_{blocks} Mamba blocks with residual connections and RMS normalization. The forward encoder processes the embedded trace left-to-right, and the backward encoder processes the flipped trace right-to-left. The last hidden state of each encoder therefore summarizes the sequence from opposite ends: the forward state has seen the whole trace up to its final position, the backward state up to its first. Concatenating the two yields a $2D_{model}$-dimensional representation in which every position contributes context from both directions.

Classification module. A single linear layer maps the concatenated representation to logits over the 256 classes.

4.4 Training and Evaluation Protocol

The 50 000 profiling traces are split into 40 000 for training and 10 000 for validation. Both models are trained for up to 100 epochs with the Adam optimizer, a batch size of 768, a learning rate of $5{\times}10^{-3}$, and weight decay of 0.01.

Model selection is performed on validation guessing entropy rather than on validation accuracy or loss, as previous work shows that GE is a better validation and evaluation metric [16, 18]. The model weights of the epoch with the lowest validation GE are retained.

Table 1: Model hyperparameters.

Model	Parameters	Configuration
MLP	155,000	$N_{neurons}$=128, N_{hidden}=2
Mamba	148,000	D_{model}=64, N_{blocks}=2

The selected model is then evaluated in three ways. We report the key rank obtained when all 10 000 attack traces are used, which indicates whether the attack succeeds at all. We report the mean guessing entropy over 100 independent attack simulations, each performed on a random subsample of 1 000 attack traces, which indicates how consistently it succeeds. Finally, we report GE convergence over those 1 000 traces, which indicates how many traces the attack needs in practice.

4.5 Model Hyperparameter Search

All configurations below were compared on validation guessing entropy, using the protocol of Section 4.4.

Convolution module: we searched kernel sizes $k \in [3, 5]$ and strides $s \in [1, 3, 5]$. Based on the results, we use $k = 3$ and $s = 3$, which gives non-overlapping windows and a sequence length of $L = 233$. Configurations with a stride smaller than the kernel size did not improve attack performance, and neither did deeper convolutional front-ends with ReLU activations and batch normalization.

Mamba module: we searched the model dimension $D_{model} \in [32, 64]$ and the number of stacked blocks per direction $N_{blocks} \in [2, 4]$. Based on the results, we use $D_{model} = 64$ and $N_{blocks} = 2$, that is, four Mamba blocks in total, two per direction.

MLP baseline: we fix $N_{hidden} = 2$ following Perin et al. [16] and set $N_{neurons}$ so that the parameter count approximately matches that of the tuned Mamba model. The two resulting configurations are given in Table 1.

5 Results

We evaluate both models on ASCADv1 following the protocol of Section 4.4. Recall that an attack is effective when the guessing entropy reaches zero, and that it is more efficient when it reaches zero with fewer attack traces. If GE instead stalls at a value x, the adversary is left with a residual search over the remaining candidates rather than with a direct recovery of the byte. We first report the validation GE on which the models were selected, and then the attack performance of the selected models.

Figure 2 shows validation GE as a function of the training epoch. The two models reach their best epoch at almost the same point — epoch 15 for the MLP and epoch 14 for Mamba — but they behave differently on either side of it. Over the first ten epochs, Mamba decreases faster, which indicates that the selective state-space encoder extracts usable leakage from fewer passes over the training set. From there on, only the MLP reaches a validation GE of 0.0, and it holds that value for the remainder of training. Mamba settles at a best validation GE of 0.92.

Table 2 summarizes the attack results of the selected models. Both architectures recover the target key byte: using all 10 000 attack traces, both reach a key rank of 0, meaning the correct byte is the first candidate and no additional search is needed. Both models thus

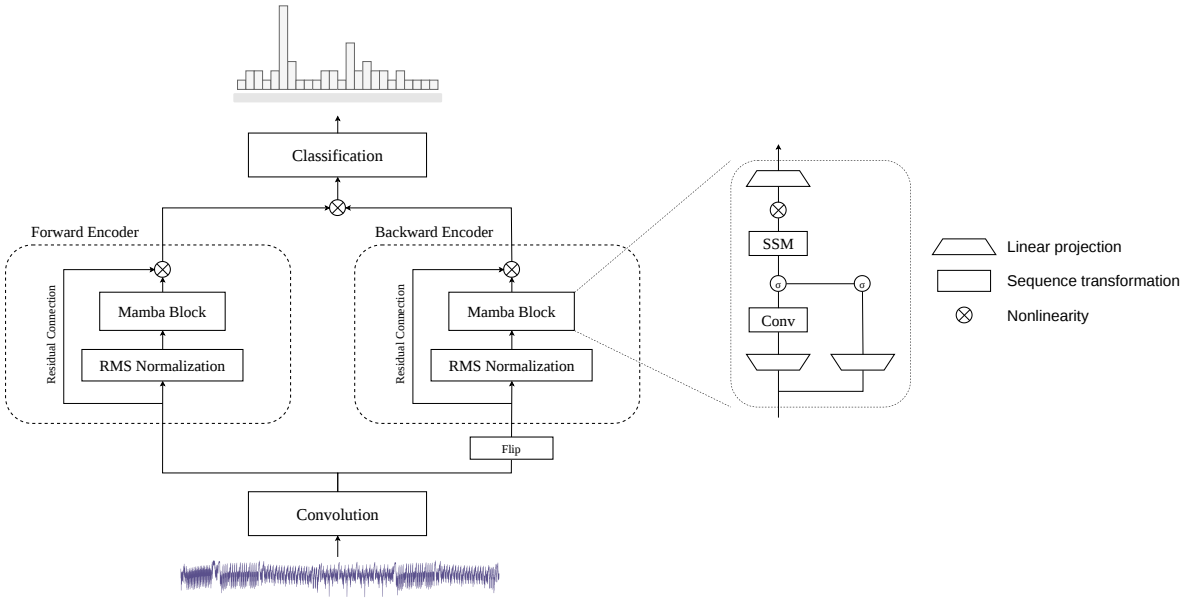


Figure 1: Mamba model architecture. A 1-D convolution embeds the raw trace into a sequence of L vectors of dimension D_{model}, which is processed by a forward and a backward encoder of N_{blocks} Mamba blocks each. Their final hidden states are concatenated, giving a representation with context from both directions, and a linear layer maps it to logits over the 256 classes.

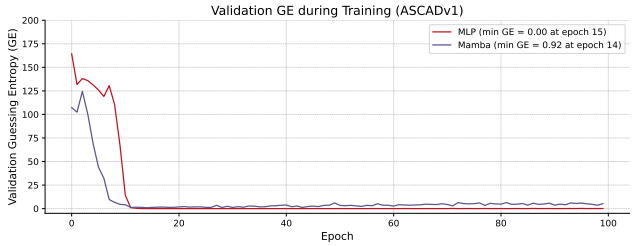


Figure 2: Validation guessing entropy per training epoch on ASCADv1. MLP - red curve, Mamba - purple curve. The epoch with the lowest validation GE is retained for the attack: epoch 15 for the MLP (GE = 0.00) and epoch 14 for Mamba (GE = 0.92).

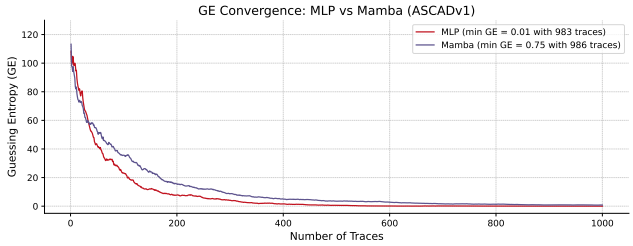


Figure 3: Guessing entropy convergence over all attack traces. MLP - red curve, Mamba - purple curve. Both models recover the key within roughly 1000 traces, though MLP reaches a lower GE (0.01 at 983 traces) than Mamba (0.75 at 986 traces).

succeed against ASCADv1 despite its first-order masking, which confirms that a selective state-space encoder can learn to recombine the mask shares from the leakage alone.

The two models differ in how consistently they achieve this. Over 100 attack simulations, each on a random subsample of 1 000 attack traces, the MLP obtains a mean GE of 0.0, meaning that every one of the 100 simulated attacks placed the correct byte first. Mamba obtains a mean GE of 0.04. Since its median GE is also 0.0, this difference is not a general shift but the effect of a small number of simulations in which the correct byte was displaced from the top position. In practice the distinction is minor — an adversary in those runs tests one additional candidate — but it indicates that the Mamba model is marginally more sensitive to to the given traces.

Figure 3 shows GE against the number of attack traces, computed in a single pass over the full attack set rather than averaged over subsamples. The two GE numbers therefore answer different questions: the mean GE reported above says how often an attack of a given size succeeds, while the convergence curve says how one attack behaves as traces accumulate. Mamba decreases marginally faster over roughly the first 50 traces, consistent with its faster progress during early training. Beyond that point the ordering reverses: the MLP settles at a low value sooner and holds it, at 0.01 after 983 traces, whereas Mamba is still at 0.75 after 986 traces.

6 Discussion

On ASCADv1, both models achieve perfect key rank, but the MLP is more consistent (mean GE 0.0 compared to 0.04) and converges to a stable low GE slightly faster in absolute trace count, even

Table 2: Model performance on ASCADv1.

Model	Key Rank	Median GE	Mean GE	GE Convergence
MLP	0	0.0	0.0	0.01 @ 983 traces
Mamba	0	0.0	0.04	0.75 @ 986 traces

though Mamba trains faster in early epochs. This suggests that for a relatively simple, fixed-key, feature-selected, masked dataset such as ASCADv1, a well-tuned MLP already captures the exploitable leakage, and Mamba's added architectural complexity yields no measurable attack-effectiveness gain at matched parameter count.

7 Limitations & Outlook

Real-world traces often suffer from misalignment, and feature selection may be infeasible when traces are collected without knowledge of the secret key. It would therefore be valuable to evaluate Mamba on raw, unaligned traces without pre-selected features and in the presence of noise; it remains an open question how well either model learns from such harder traces. Datasets with stronger countermeasures such as affine masking and shuffling, as well as targets implementing different cryptographic algorithms, would further test whether Mamba's sequence-modeling capacity offers an advantage where simpler models struggle. Finally, although we explored several convolution and Mamba hyperparameters, we did not apply automated hyperparameter optimization [24], which an exhaustive search could build on to yield stronger models.

8 Conclusion

We proposed a bidirectional Mamba-based network for deep learning-based SCA, stacking residual Mamba blocks to model leakage in both directions of a power trace. On masked AES traces, the model recovers the key with rank zero and fast GE convergence, showing that selective state-space models can learn meaningful representations from side-channel data. At matched parameter count, however, it did not outperform an MLP baseline, which converged faster and more consistently, suggesting that on simple fixed-key datasets an MLP already captures the exploitable leakage. Mamba's benefits may emerge on longer, noisier and more strongly protected traces, which we leave to future work.

References

[1] Amjed Abbas Ahmed and Mohammad Kamrul Hasan. 2023. Multi-Layer Perceptrons and Convolutional Neural Networks Based Side-Channel Attacks on AES Encryption. In *2023 International Conference on Engineering Technology and Technopreneurship (ICE2T)*. 69–73. doi:10.1109/ICE2T58637.2023.10540465

[2] Amjed Abbas Ahmed, Mohammad Kamrul Hasan, Imran Memon, Azana Hafizah Mohd Aman, Shayla Islam, Thippa Reddy Gadekallu, and Sufyan Ali Memon. 2024. Secure AI for 6G Mobile Devices: Deep Learning Optimization Against Side-Channel Attacks. *IEEE Transactions on Consumer Electronics* 70, 1 (2024), 3951–3959. doi:10.1109/TCE.2024.3372018

[3] Ryad Benadjila, Emmanuel Prouff, Rémi Strullu, Eleonora Cagli, and Cécile Dumas. 2020. Deep learning for side-channel analysis and introduction to ASCAD database. *Journal of Cryptographic Engineering* 10 (06 2020). doi:10.1007/s13389-019-00220-8

[4] Suresh Chari, Josyula R. Rao, and Pankaj Rohatgi. 2003. Template Attacks. In *Cryptographic Hardware and Embedded Systems - CHES 2002*, Burton S. Kaliski, çetin K. Koç, and Christof Paar (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 13–28.

[5] Julien Doget, Emmanuel Prouff, Matthieu Rivain, and François-Xavier Standaert. 2011. Univariate side channel attacks and leakage modeling. *Journal of Cryptographic Engineering* 1, 2 (2011), 123–144. doi:10.1007/s13389-011-0010-2

[6] T. Feng, H. Gao, X. Li, et al. 2025. Side-Channel Attacks on Convolutional Neural Networks Based on the Hybrid Attention Mechanism. *Discover Applied Sciences* 7, 390 (2025). doi:10.1007/s42452-025-06854-0

[7] Albert Gu and Tri Dao. 2024. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. In *First Conference on Language Modeling*. https://openreview.net/forum?id=tEYskw1VY2

[8] Pengfei He, Ying Zhang, Han Gan, Jianfei Ma, and Hongxin Zhang. 2024. Side-channel attacks based on attention mechanism and multi-scale convolutional neural network. *Computers and Electrical Engineering* 119 (2024), 109515. doi:10.1016/j.compeleceng.2024.109515

[9] J.H. Kim, Stjepan Picek, Annelie Heuser, Shivam Bhasin, and Alan Hanjalic. 2019. Make Some Noise: Unleashing the Power of Convolutional Neural Networks for Profiled Side-channel Analysis. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2019, 3 (2019), 148–179. doi:10.13154/tches.v2019.i3.148-179

[10] Paul C. Kocher. 1996. Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems. In *Advances in Cryptology — CRYPTO '96*, Neal Koblitz (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 104–113.

[11] Paul C. Kocher, Joshua Jaffe, and Benjamin Jun. 1999. Differential Power Analysis. In *Advances in Cryptology — CRYPTO '99 (Lecture Notes in Computer Science, Vol. 1666)*, Michael J. Wiener (Ed.). Springer, Santa Barbara, California, USA, 388–397.

[12] Zhaobin Li, Chenchong Du, and Xiaoyi Duan. 2025. Efficient AES Side-Channel Attacks Based on Residual Mamba Enhanced CNN. *Entropy* 27, 8 (2025). doi:10.3390/e27080853

[13] Xiangjun Lu, Chi Zhang, Pei Cao, Dawu Gu, and Haining Lu. 2021. Pay Attention to Raw Traces: A Deep Learning Architecture for End-to-End Profiling Attacks. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2021, 3 (Jul. 2021), 235–274. doi:10.46586/tches.v2021.i3.235-274

[14] Houssem Maghrebi, Thibault Portigliatti, and Emmanuel Prouff. 2016. Breaking Cryptographic Implementations Using Deep Learning Techniques. In *Security, Privacy, and Applied Cryptography Engineering*, Claude Carlet, M. Anwar Hasan, and Vishal Saraswat (Eds.). Springer International Publishing, Cham, 3–26.

[15] National Institute of Standards, Technology (NIST), Morris J. Dworkin, Elaine Barker, James Nechvatal, James Foti, Lawrence E. Bassham, E. Roback, and James Dray Jr. 2001. Advanced Encryption Standard (AES). doi:10.6028/NIST.FIPS.197

[16] Guilherme Perin, Lichao Wu, and Stjepan Picek. 2022. Exploring Feature Selection Scenarios for Deep Learning-based Side-channel Analysis. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2022, 4 (Aug. 2022), 828–861. doi:10.46586/tches.v2022.i4.828-861

[17] Stjepan Picek, Annelie Heuser, Alan Jovic, and Lejla Batina. 2019. A Systematic Evaluation of Profiling Through Focused Feature Selection. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 27, 12 (2019), 2802–2815. doi:10.1109/TVLSI.2019.2937365

[18] Stjepan Picek, Annelie Heuser, Alan Jovic, Shivam Bhasin, and Francesco Regazzoni. 2018. The Curse of Class Imbalance and Conflicting Metrics with Machine Learning for Side-channel Evaluations. *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2019, 1 (Nov. 2018), 209–237. doi:10.13154/tches.v2019.i1.209-237

[19] Stjepan Picek, Annelie Heuser, Guilherme Perin, and Sylvain Guilley. 2022. Profiled Side-Channel Analysis in the Efficient Attacker Framework. In *Smart Card Research and Advanced Applications*, Vincent Grosso and Thomas Pöppelmann (Eds.). Springer International Publishing, Cham, 44–63.

[20] Stjepan Picek, Guilherme Perin, Luca Mariot, Lichao Wu, and Lejla Batina. 2023. SoK: Deep Learning-based Physical Side-channel Analysis. *ACM Comput. Surv.* 55, 11, Article 227 (Feb. 2023), 35 pages. doi:10.1145/3569577

[21] Jean-Jacques Quisquater and David Samyde. 2001. ElectroMagnetic Analysis (EMA): Measures and Counter-measures for Smart Cards. In *Smart Card Programming and Security*, Isabelle Attali and Thomas Jensen (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 200–210.

[22] François-Xavier Standaert. 2010. Introduction to Side-Channel Attacks. In *Secure Integrated Circuits and Systems*, Ingrid M.R. Verbauwhede (Ed.). Springer US, Boston, MA, 27–42. doi:10.1007/978-0-387-71829-3_2

[23] S. Swaminathan, Ł. Chmielewski, G. Perin, and S. Picek. 2022. Deep Learning-Based Side-Channel Analysis Against AES Inner Rounds. In *Applied Cryptography and Network Security Workshops (ACNS 2022) (Lecture Notes in Computer Science, Vol. 13285)*. Springer, Cham. doi:10.1007/978-3-031-16815-4_10

[24] Lichao Wu, Guilherme Perin, and Stjepan Picek. 2024. I Choose You: Automated Hyperparameter Tuning for Deep Learning-Based Side-Channel Analysis. *IEEE Transactions on Emerging Topics in Computing* 12, 2 (2024), 546–557. doi:10.1109/TETC.2022.3218372

[25] Lichao Wu, Léo Weissbart, Marina Krček, Huimin Li, Guilherme Perin, Lejla Batina, and Stjepan Picek. 2023. Label Correlation in Deep Learning-Based Side-Channel Analysis. *Trans. Info. For. Sec.* 18 (Jan. 2023), 3849–3861. doi:10.1109/TIFS.2023.3287728